計算機アプリの仕組み

simple-calculator / 初学者向け解説（useState・型・部品の役割）

1. 全体像

画面の部品と、裏で覚えている「4つのメモ」が分かれています。

見える部分

Display（画面）

Button（0〜9, +, =, AC…）

見えないメモ（useState）

displayValue

previousValue

operator

waitingForOperand

計算用

calculate.ts

a と b を計算して結果を返す

ボタンを押す → App.tsx がメモを書き換え → Display が新しい値を表示。必要なら calculate を呼ぶ。

2. useState は「メモ用紙」

1枚のメモのイメージ

今の値

`displayValue = "7"`

書き換える道具

`setDisplayValue`

ボタンを押す → `setDisplayValue(...)` でメモを書く → React が画面を描き直す

言葉	意味
`displayValue`	今メモに書いてある内容
`setDisplayValue(...)`	メモを書き換える
書き換えたあと	React が画面を描き直す

このアプリではメモが4枚あります。

メモ	例	役割
`displayValue`	`"15"`	画面に出す文字列
`previousValue`	`7`	演算子の前の数
`operator`	`"+"`	どの計算か
`waitingForOperand`	`true` / `false`	演算子の直後か

3. 例：7 + 8 = のとき

> **① 7 を押す**
> 画面メモだけ `"7"`

> **② + を押す**
> 前の数に `7` 、演算子に `"+"` 、待ちをオン（`waitingForOperand = true`）

> **③ 8 を押す**
> 待ちオンなので末尾に足さず、画面を `"8"` に切り替え。待ちをオフ

> **④ = を押す**
> `calculate(7, 8, "+")` → `15` 。画面に出して、計算用メモをリセット

4. 型は「門番」




```
type Operator = "+" | "−" | "×" | "÷";
```

意味は、「演算子はこの4文字だけOK」です。

- 通る例：`chooseoperator("+")`
- 止まる例：`chooseoperator("*")`（キーボードの * は型に無い）

`digit: string` なのは、数字も画面も**文字としてつなぐ**ためです。

- `"1" + "2"` → `"12"`（文字列の連結）
- `1 + 2` → `3`（数値の足し算）

5. 部品の役割

ファイル / 部品	役割
`App.tsx`	司令塔。メモの持ち主。押されたらどうするか
`Display`	渡された文字を表示する
`Button`	見た目とクリックの受け皿
`calculate.ts`	足す・引くなどの計算だけ
`types.ts`	許す値のルール（型）

一番大事な一文
ボタンを押す → `set〜` でメモを書き換える → 画面が新しいメモを表示する。
型はそのメモや引数に変な値が入らないように見張る。